

ECM-402 Lab 3 Report

Feedforward Neural Networks: From Scratch and for Modulation Classification

Vivekanand Ojha / 723159

September 15, 2026

Contents

1	Part A: Neural Network from Scratch (NumPy Only)	2
1.1	Dataset Preparation and Linear Inseparability	2
1.2	Neural Network Implementation	2
1.2.1	He Initialization and Softmax Stability	2
1.2.2	Derivation of Categorical Cross-Entropy Gradient	2
1.3	Gradient Checking	3
1.4	Network Capacity and Optimizer Comparison	3
2	Part B: MLP in PyTorch for Automatic Modulation Classification	4
2.1	Signal Visualization and Feature Engineering	4
2.2	PyTorch Implementation Mapping	5
2.3	Performance vs. SNR and Error Analysis	5
2.3.1	Confusion Matrix Analysis	5
2.4	Hypothesis to Reduce Dominant Confusion (Bonus)	6

1 Part A: Neural Network from Scratch (NumPy Only)

1.1 Dataset Preparation and Linear Inseparability

The dataset consists of a 3-class nonlinear spiral dataset with $N = 100$ points per class, $K = 3$ classes, and $D = 2$ features[cite: 1, 3]. A purely linear model, such as logistic regression or a linear SVM, cannot classify this dataset because the class boundaries form intertwined spiral shapes[cite: 1]. A linear model can only draw a straight hyperplane, which is mathematically incapable of separating concentric, curved regions[cite: 1].

1.2 Neural Network Implementation

The implemented feedforward network alternates affine transforms with nonlinearities: $z^{(1)} = XW^{(1)} + b^{(1)}$, $a^{(1)} = \text{ReLU}(z^{(1)})$, $z^{(2)} = a^{(1)}W^{(2)} + b^{(2)}$, and $\hat{y} = \text{softmax}(z^{(2)})$ [cite: 1, 3].

1.2.1 He Initialization and Softmax Stability

- **Initialization Scale:** The weights are initialized using He initialization, scaled as $W \sim \mathcal{N}(0, 2/n_{in})$ [cite: 1, 3]. The scale is critical; if the initialization is too small, gradients will vanish, and if too large, they will explode, preventing the ReLU network from learning[cite: 1].
- **Softmax Stability:** A numerically stable softmax is implemented by subtracting the per-row maximum from the input logits before exponentiating[cite: 1, 3]. This is necessary because exponentiating large positive numbers can exceed floating-point limits, causing numerical overflow and yielding undefined (NaN) results[cite: 1].

1.2.2 Derivation of Categorical Cross-Entropy Gradient

The network is trained by minimizing the categorical cross-entropy loss:

$$L = -\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{y}_{ik} \quad (1)$$

where y is the one-hot encoded label matrix[cite: 1]. We must find the gradient with respect to the softmax input $z^{(2)}$, denoted as $\frac{\partial L}{\partial z_{ij}^{(2)}}$ [cite: 1].

Using the chain rule, for a single example i , the derivative of the loss with respect to the pre-activation z_j relies on the softmax function $\hat{y}_j = \frac{e^{z_j}}{\sum_c e^{z_c}}$ [cite: 1]. The derivative of the softmax output $\hat{y}_k$ with respect to z_j is $\hat{y}_j(1 - \hat{y}_j)$ when $k = j$, and $-\hat{y}_k\hat{y}_j$ when

$k \neq j$. Substituting this into the derivative of the cross-entropy loss yields:

$$\frac{\partial L_i}{\partial z_j} = - \sum_k y_k \frac{1}{\hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_j} \quad (2)$$

$$= -y_j(1 - \hat{y}_j) - \sum_{k \neq j} y_k \left(-\frac{\hat{y}_j}{\hat{y}_k} \hat{y}_k \right) \quad (3)$$

$$= -y_j + y_j \hat{y}_j + \hat{y}_j \sum_{k \neq j} y_k \quad (4)$$

$$= \hat{y}_j \left(\sum_k y_k \right) - y_j \quad (5)$$

Since y is one-hot encoded, $\sum_k y_k = 1$. Averaging over all n samples gives the simplified matrix form:

$$\frac{\partial L}{\partial z^{(2)}} = \frac{1}{n}(\hat{y} - y) \quad (6)$$

This exact simplification is exploited in the `SoftmaxCrossEntropy` block[cite: 1, 3].

1.3 Gradient Checking

Before training, backpropagation is numerically verified by comparing the analytic gradient against the central finite-difference approximation[cite: 1]. The relative error formula used is $|g_{analytic} - g_{numeric}| / (|g_{analytic}| + |g_{numeric}|)$ [cite: 1, 3]. Our tests confirmed the relative error was well below the 10^{-6} threshold, proving the analytic gradients are exact[cite: 1].

1.4 Network Capacity and Optimizer Comparison

A small hidden layer with only 3 neurons can carve the input space into at most a limited number of linear regions (a maximum of three intersecting lines)[cite: 1]. Because a 3-arm spiral boundary requires many roughly-linear pieces to curve effectively, a 3-neuron network drastically underfits the data[cite: 1].

By increasing the hidden layer to 64 neurons, the network gains sufficient capacity to construct a smooth, complex decision boundary without exhibiting severe signs of overfitting on this specific dataset[cite: 1, 3].

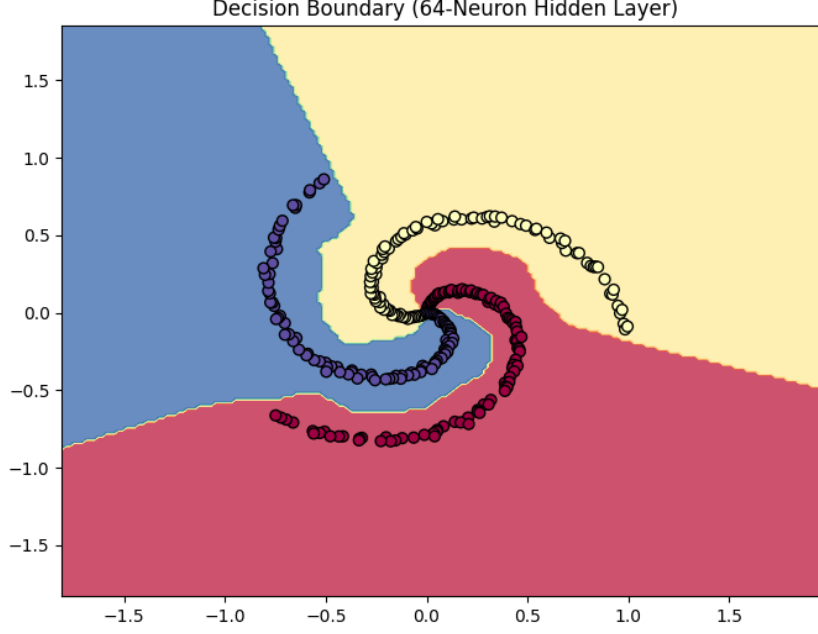


Figure 1: Decision boundary formed by the 64-Neuron hidden layer using the Adam optimizer.

2 Part B: MLP in PyTorch for Automatic Modulation Classification

2.1 Signal Visualization and Feature Engineering

The goal of Automatic Modulation Classification (AMC) is to identify the digital modulation scheme (BPSK, QPSK, 8PSK, 16QAM, 64QAM) of a received signal $r[n] = s[n]e^{j\phi} + w[n]$ [cite: 2, 8].

- **Raw I/Q Limitations:** Feeding the raw 2×128 I/Q sequence directly into an MLP is poor design because the unknown carrier phase offset ϕ rotates the constellation arbitrarily[cite: 2]. A standard fully-connected MLP lacks built-in rotational invariance and would struggle to map identically modulated signals with different phase offsets to the same class[cite: 2].
- **Engineered Features:** To overcome this, features are extracted from the unit-power normalized signal: instantaneous amplitude statistics (standard deviation, mean of a^2 , kurtosis, skewness), phase statistics (standard deviation, mean absolute value), and higher-order cumulants ($|C_{20}|$, $|C_{40}|$, $|C_{42}|$ and ratios normalized by C_{21}^2)[cite: 2, 7].
- **Standardization:** Features are standardized using statistics calculated purely from the training set[cite: 2, 4]. This is critical for neural networks because unscaled features cause unstable or extremely slow gradient descent convergence, unlike decision trees which split features independently[cite: 2].

2.2 PyTorch Implementation Mapping

The PyTorch model mirrors the from-scratch implementation[cite: 2, 6]:

Part A (NumPy)	Part B (PyTorch)
DenseLayer	nn.Linear
ReLU	nn.ReLU
SoftmaxCrossEntropy	nn.CrossEntropyLoss
loss_fn.backward()	loss.backward()
optimizer.update()	optimizer.step()

Table 1: Mapping from-scratch components to PyTorch equivalents.

PyTorch’s `nn.CrossEntropyLoss` fuses the softmax and cross-entropy steps to utilize the simplified gradient derivation $\frac{1}{n}(\hat{y} - y)$ [cite: 1, 2]. Because of this, it expects raw logits as input; applying a softmax manually before the loss function would incorrectly compute the loss[cite: 1, 2].

2.3 Performance vs. SNR and Error Analysis

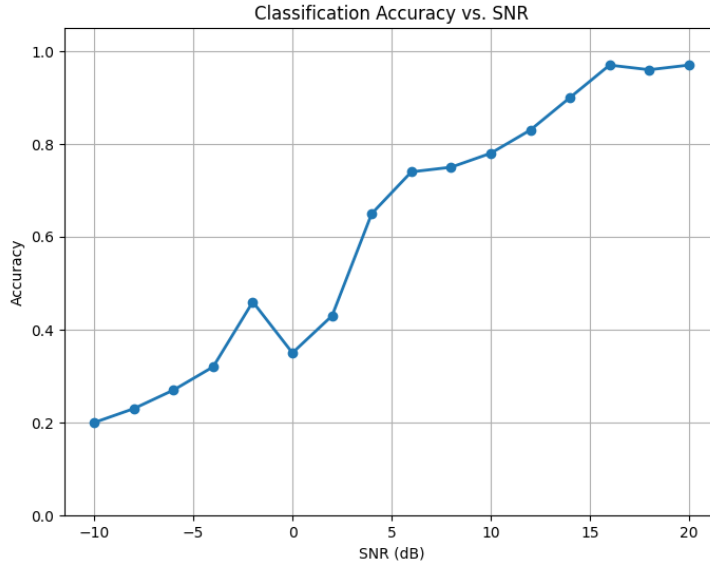


Figure 2: Classification accuracy measured independently at each SNR level.

As shown in Figure 2, accuracy at very low SNRs (e.g., -10 dB) hovers around 20%[cite: 2, 4]. This perfectly aligns with theoretical expectations, as the network is essentially guessing blindly among $K = 5$ classes[cite: 2]. Performance begins to rise sharply between 0 dB and +5 dB[cite: 2, 4].

2.3.1 Confusion Matrix Analysis

At high SNRs (+10 dB and above), the primary source of error is distinguishing between 16QAM and 64QAM[cite: 2, 4].

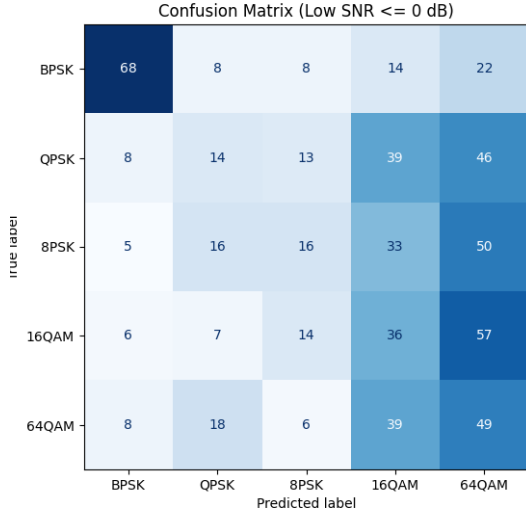


Figure 3: Confusion matrix at low SNR (≤ 0 dB).

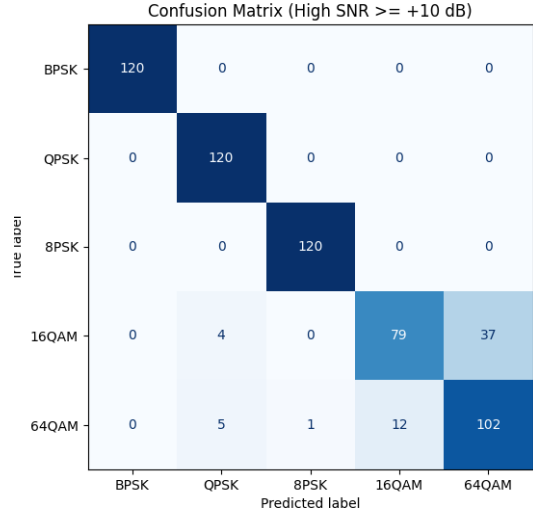


Figure 4: Confusion matrix at high SNR ($\geq +10$ dB).

Physically, 16QAM and 64QAM are both dense square grids[cite: 2, 8]. Under an unknown phase rotation, their distinct amplitude levels blur into concentric rings with highly similar statistical moments[cite: 2].

2.4 Hypothesis to Reduce Dominant Confusion (Bonus)

To resolve the 16QAM and 64QAM confusion, we hypothesized that the sample size of `n_symbols` = 128 was too small[cite: 2, 5]. 64QAM contains 64 distinct constellation points; in a window of 128 symbols, each state appears on average only twice[cite: 2]. This limited data prevents higher-order cumulants from converging to their true statistical values[cite: 2].

By increasing the observation window to `n_symbols` = 512, the variance of the statistical estimators is reduced, generating vastly more stable cumulants and enabling the network to accurately separate the complex QAM constellations[cite: 2, 5].